PMVC-WNM: The Co-Training Function

Technical Report on the Core Training Algorithm

Phonetic Multi-View Co-training with Weakly Supervised Noise Model

CSE 4622: Machine Learning Lab | Islamic University of Technology

Prototype notebook: PMVC_WNM_Banglish_Classification.ipynb

1. What the Co-Training Function Does

The prototype's entire learning algorithm lives in a single function, `co_training()`. It implements the classic two-view, semi-supervised co-training procedure (Blum & Mitchell, 1998) applied to Banglish sentiment classification, extended with a lightweight **Weakly Supervised Noise Model (WNM)** that reflects the project proposal's "Adaptive Noise Injection" idea.

Two classifiers are trained on two independent feature views of the same text:

- **View A (orthographic):** a linear-kernel SVM (`f_A`) trained on character 3/4-gram TF-IDF features — captures spelling and typos.
- **View B (phonetic):** a Logistic Regression model (`f_B`) trained on TF-IDF features of a rule-based Banglish phonetic encoding — captures pronunciation, independent of spelling.

Starting from a small labeled seed set L, the two classifiers iteratively "teach" each other by pseudo-labeling a larger pool of unlabeled data U: in each round, whichever view is most confident about an unlabeled example contributes a pseudo-label that both views then train on. Because the two views make largely independent errors (a spelling mistake rarely also changes how a word sounds), disagreement between them is a signal of noisy/ambiguous input — the WNM step turns that signal into a per-class reliability score used to down-weight untrustworthy pseudo-labels when the character view is retrained.

2. Mathematical Formulation

2.1 Feature views

Let x be a cleaned Banglish sentence and φ(x) its phonetic encoding (consonant-class folding, e.g. *khacchi*, *khacci*, *khachi* → the same code). The two views are:

$$X_A = \mathrm{TFIDF}_{\mathrm{char}(3,4)}(x), \quad \dim = 5000$$

$$X_B = \mathrm{TFIDF}_{\mathrm{char}(2,3)}(\varphi(x)), \quad \dim = 3000$$

2.2 Confidence and prediction, per view

For classifier $f \in \{f_A, f_B\}$ and an unlabeled example x, the predicted class and the model's confidence in that prediction are:

$$\hat{y}(x) = \operatorname{argmax}_k \, P(y = k \mid x \,;\, f)$$

$$c(x) = \max_k \, P(y = k \mid x \,;\, f)$$

2.3 Cross-teaching selection rule

At iteration t, let U_{rem} be the unlabeled examples not yet pseudo-labeled. The g most confident examples from each view (g = growth_per_iter) are proposed, and only those clearing the confidence threshold τ are actually accepted:

$$S_A = \text{top-}g_{x \in U_{\mathrm{rem}}} \; c_A(x) \;, \quad S_B = \text{top-}g_{x \in U_{\mathrm{rem}}} \; c_B(x)$$

$$S = \{ x \in (S_A \cup S_B) : \max(c_A(x), c_B(x)) \geq \tau \}$$

$$\hat{y}(x) = \hat{y}_A(x) \quad \text{if } c_A(x) \geq c_B(x) \quad \text{else } \hat{y}_B(x), \quad \text{for } x \in S$$

Both views' training pools are then grown with the newly pseudo-labeled examples, and the loop repeats for T = n_iterations (T = 10, matching the proposal's ST = 10S convergence criterion) or until S = ∅:

$$L_A \leftarrow L_A \cup \{(x, \hat{y}(x)) : x \in S\}, \quad L_B \leftarrow L_B \cup \{(x, \hat{y}(x)) : x \in S\}, \quad U \leftarrow U - S$$

> *Implementation note:* the proposal describes strict cross-teaching (only f_A's confident predictions feed f_B, and vice versa). This prototype uses a simplified variant where the single more-confident prediction for each selected example is shared into **both** pools — mathematically a special case of cross-teaching, and considerably simpler to implement and verify at prototype scale.

2.4 Weakly Supervised Noise Model (WNM)

The proposal's noise transition matrix (context.txt, Slide 5) models the probability of an observed (spelling-distorted) phonetic class given the true one:

$$P(C_{obs} \mid P_{true}) = \text{count}(C_{obs}, P_{true}) \,/\, \Sigma_k \,\text{count}(C_k, P_{true})$$

The prototype approximates this with a cheaper **per-class agreement rate**: for every pseudo-labeling round, each unlabeled example's phonetic-view prediction $\hat{y}_B(x)$ is checked against the character view's prediction. This tallies agree/disagree counts per class c, giving a reliability score:

$$R(c) = \text{agree}(c) \,/\, (\text{agree}(c) + \text{disagree}(c)) \quad \in [0, 1]$$

R(c) is then used to build a per-example sample weight when the character-view classifier f_A is retrained — seed-labeled examples keep full weight, and pseudo-labeled examples are trusted in proportion to how reliable their class has historically been:

$$w_i = 1 \quad \text{if } x_i \in L \text{ (seed)}, \quad w_i = 0.5 + 0.5 \cdot R(\hat{y}_i) \quad \text{if } x_i \text{ pseudo-labeled}$$

$$f_A \leftarrow \text{fit}(X_A[L \cup S],\ y[L \cup S]\ ;\ \text{sample\_weight} = w)$$

> This is exactly the "disagreement between f_A and f_B signals spelling distortions" idea from the proposal, implemented as label-noise-aware sample re-weighting rather than a full confusion-style transition matrix.

3. Python Implementation (Co-Training Function Only)

This is the exact, unmodified `co_training()` function from Section 7/8 of PMVC_WNM_Banglish_Classification.ipynb, plus the two calls that invoke it (standard co-training and the full PMVC-WNM run).

```
def co_training(XA_L, XB_L, y_L, XA_U, XB_U,
                n_iterations=10, confidence_threshold=0.6, growth_per_iter=40,
                use_noise_model=False, random_state=RANDOM_STATE):
    # confidence_threshold lowered from the proposal's 0.85 to 0.6: with only
    # N_LABELED=300 prototype seed samples the classifiers are less confident
    # early on, and 0.85 was tight enough to stall pseudo-labeling after 1-2
    # iterations. 0.6 keeps the loop actively growing across all 10 iterations
    # so the training progress is visible.
    """
    PMVC co-training loop over two independent views.
    Returns fitted (f_A, f_B), plus, if use_noise_model=True, the learned
    disagreement-based sample-weighting used for the noise-aware retrain of f_A.
    """
    XA_L_cur, XB_L_cur, y_L_cur = XA_L, XB_L, list(y_L)
    remaining = list(range(XA_U.shape[0]))

    f_A = SVC(kernel="linear", probability=True, random_state=random_state)
    f_B = LogisticRegression(max_iter=1000, random_state=random_state)

    disagreement_counts = {}  # phonetic_class_pred -> [agree, disagree] for the noise matrix

    for it in range(n_iterations):
        if not remaining:
            break
        f_A.fit(XA_L_cur, y_L_cur)
        f_B.fit(XB_L_cur, y_L_cur)

        XA_rem = XA_U[remaining]
        XB_rem = XB_U[remaining]

        proba_A = f_A.predict_proba(XA_rem)
        proba_B = f_B.predict_proba(XB_rem)
        classes_A, classes_B = f_A.classes_, f_B.classes_

        pred_A = classes_A[np.argmax(proba_A, axis=1)]
        pred_B = classes_B[np.argmax(proba_B, axis=1)]
        conf_A = proba_A.max(axis=1)
        conf_B = proba_B.max(axis=1)

        # Track disagreement for the noise model (WNM)
        for pa, pb in zip(pred_A, pred_B):
            disagreement_counts.setdefault(pb, [0, 0])
            if pa == pb:
                disagreement_counts[pb][0] += 1
            else:
                disagreement_counts[pb][1] += 1

        # f_A's most confident predictions get added to f_B's pool, and vice versa
        top_A_idx = np.argsort(-conf_A)[:growth_per_iter]
        top_B_idx = np.argsort(-conf_B)[:growth_per_iter]
        chosen_local = set(top_A_idx.tolist()) | set(top_B_idx.tolist())
        chosen_local = [i for i in chosen_local if max(conf_A[i], conf_B[i]) >= confidence_threshold]

        if not chosen_local:
            break

        chosen_global = [remaining[i] for i in chosen_local]
        pseudo_labels = [pred_A[i] if conf_A[i] >= conf_B[i] else pred_B[i] for i in chosen_local]

        from scipy.sparse import vstack
        XA_L_cur = vstack([XA_L_cur, XA_U[chosen_global]])
        XB_L_cur = vstack([XB_L_cur, XB_U[chosen_global]])
        y_L_cur = y_L_cur + pseudo_labels

        remaining = [i for i in remaining if i not in set(chosen_global)]
        print(f"  iter {it+1:2d}: +{len(chosen_global)} pseudo-labeled, "
              f"{len(remaining)} left in U, train size = {len(y_L_cur)}")

    noise_matrix = None
```

```
    if use_noise_model:
        noise_matrix = {
            cls: (agree / max(agree + disagree, 1))
            for cls, (agree, disagree) in disagreement_counts.items()
        }
        # Re-weight and retrain f_A, down-weighting classes that showed high
        # A/B disagreement (i.e. likely spelling-noise-corrupted regions),
        # which is the "Adaptive Noise Injection" step from the proposal.
        sample_weight = np.ones(len(y_L_cur))
        base_labels = list(y_L)
        for i, lab in enumerate(y_L_cur):
            if i >= len(base_labels):  # pseudo-labeled example
                reliability = noise_matrix.get(lab, 1.0)
                sample_weight[i] = 0.5 + 0.5 * reliability  # in [0.5, 1.0]
        f_A.fit(XA_L_cur, y_L_cur, sample_weight=sample_weight)

    return f_A, f_B, noise_matrix


# --- Standard co-training baseline (no noise modeling) ---
fA_std, fB_std, _ = co_training(XA_L, XB_L, y_L, XA_U, XB_U, use_noise_model=False)
pred_std = fA_std.predict(XA_test)
evaluate("Standard co-training", y_test, pred_std)

# --- Full PMVC-WNM method (co-training + weakly supervised noise model) ---
fA_wnm, fB_wnm, noise_matrix = co_training(XA_L, XB_L, y_L, XA_U, XB_U, use_noise_model=True)
pred_wnm = fA_wnm.predict(XA_test)
evaluate("PMVC-WNM (ours)", y_test, pred_wnm)
```

Source: PMVC_WNM_Banglish_Classification.ipynb, Sections 7-8 (cells implementing and invoking co_training()).

4. How This Satisfies the Project Proposal

The table below maps each requirement from the proposal (context.txt) to where it is implemented in co_training() and the cells around it, so the algorithm can be traced directly back to the slide deck.

Proposal requirement (context.txt)	Implementation	Notes
“Initialization: Train f_A and f_B on small labeled set L (800 samples)” (Slide 4)	`f_A = SVC(kernel="linear")` `f_B = LogisticRegression()` `fit on XA_L_cur / XB_L_cur = L`	Prototype scales L down to 300 seed samples for speed
“Pseudo-labeling: classifiers predict labels for unlabeled set U (10,000 samples)” (Slide 4)	`proba_A = f_A.predict_proba(XA_rem)` `proba_B = f_B.predict_proba(XB_rem)`	Prototype U ≈ 2,250 samples
“Cross-Teaching: High confidence examples from f_A are added to training of f_B and vice versa” (Slide 4)	`top_A_idx / top_B_idx confidence ranking → merged into both XA_L_cur and XB_L_cur`	Simplified to a shared pool (see §2.3 implementation note)
“Convergence: Repeat for ST=10S iterations” (Slide 4)	`for it in range(n_iterations), n_iterations=10 (default)`	Matches proposal exactly
Adaptive Noise Injection / transition matrix P(C_obs|P_true) (Slide 5)	`disagreement_counts → noise_matrix reliability score → sample_weight in f_A retrain`	Approximated as a per-class agreement-rate proxy (§2.4)
View A: Character n-grams (Slide 3)	`TfidfVectorizer(char_wb, ngram(3,4), max_features=5000)`	Matches "5K N-gram Features" dataset spec
View B: Phonetic Codes / Bangla Phonetic Encoding (Slides 3-4)	`phonetic_encode_word()` `consonant-class folding + TF-IDF`	Roman-script adaptation of the Bangla consonant class table
Baseline Models for Comparison (Slide 5)	Single-view SVM, single-view RF, standard co-training (Sections 6-7)	BiLSTM baseline present but optional/off by default
Expected Performance Gain chart (Slide 6)	Section 10: macro-F1 bar chart across all methods	Prototype scores are indicative only, not the projected 88% F1

In short: the co_training() function is a directly runnable implementation of the proposal's core loop (Slide 4) plus its noise-modeling extension (Slide 5), scaled down from the proposal's 800-labeled / 10,000-unlabeled split to a faster 300-labeled / ~2,250-unlabeled prototype split so the whole thing trains and shows its per-iteration progress in well under a minute.

5. Running the Prototype on Google Colab

Step-by-step

1. Go to `colab.research.google.com` and choose **File → Upload notebook**, then select `PMVC_WNM_Banglish_Classification.ipynb` from your machine (or drag-and-drop it onto the upload dialog).

2. Once the notebook opens, click the **folder icon** in the left sidebar to open the *Files* pane (this is Colab's session storage, equivalent to the notebook's working directory).

3. Click **Upload to session storage** (the icon with an upward arrow) and select `huggingface bensentMix.csv`. This is the **only** data file the notebook reads — see the file list in Section 6 below.

4. Alternatively, upload it from a code cell instead of the Files pane:

```
from google.colab import files
uploaded = files.upload()   # pick huggingface bensentMix.csv in the file dialog
```

5. Run **Runtime → Run all**. No GPU is needed — leave the runtime type as CPU (**Runtime → Change runtime type → CPU**) since the entire pipeline is scikit-learn. Do not flip `RUN_DEEP_BASELINE` to True unless you also switch to a GPU runtime, since that optional cell needs TensorFlow.

6. All required Python packages (numpy, pandas, scikit-learn, matplotlib) are already preinstalled on Colab — no `pip install` step is required.

7. Colab's session storage is **ephemeral**: if the runtime disconnects or recycles (e.g. after being idle, or a new day), the uploaded CSV is gone and must be re-uploaded before running again.

6. Files to Upload to Colab

File	Required?	Why
`PMVC_WNM_Banglish_Classification.ipynb`	Yes	The notebook itself
`huggingface bensentMix.csv`	Yes	The only dataset the notebook actually loads (BnSentMix, Sentence/Label columns)
`requirements.txt`	No	All packages it lists are preinstalled on Colab; useful only for local setup
`BanglaSenti.csv`	No	Not read by any cell in the notebook
`BanglaSenti_SingleWords.csv`	No	Not read by any cell in the notebook
`EnBn_CodeMixed_TwoClass_Sentiment_Balanced_100k.csv`	No	Not read by any cell in the notebook
`sen_1k.csv`	No	Not read by any cell in the notebook

> The loader function reads the file by its exact local name (`load_bnsentmix_csv(path="huggingface bensentMix.csv", ...)`), so it must be uploaded with that exact filename into Colab's working directory (`/content/`, which is the default when uploaded via the Files pane or `files.upload()`).

7. Expected Training Time on Colab

Running the full notebook locally end-to-end (data load → both TF-IDF feature views → two baseline models → two full 10-iteration co-training runs, standard and PMVC-WNM → evaluation

chart) via `jupyter execute` took about **25-30 seconds** of wall-clock time, including kernel start-up. A free-tier Colab CPU runtime (2 vCPUs) is broadly comparable in raw compute to a modern laptop CPU, so at this prototype's data scale (`PROTOTYPE_SAMPLE_SIZE = 3000`, `N_LABELED = 300`) you should expect similar or slightly longer times.

Estimated total time on Colab (Runtime → Run all):

- **~1-3 minutes end-to-end**, most of it one-time overhead: Colab runtime connection/spin-up (10-30s) and package imports (a few seconds).
- The single slowest step is the linear-kernel SVM with `probability=True` (it runs internal cross-validation for probability calibration) — still well under a minute at 300-1,000 training rows.
- Both co-training loops together (20 iterations total, standard + WNM) typically finish in a few seconds once the classifiers are warmed up, since each iteration only retrains on a few hundred to ~1,000 rows.

> If you scale the constants up toward the proposal's original 800-labeled / 10,000-unlabeled split (`PROTOTYPE_SAMPLE_SIZE` and `N_LABELED` in Section 1), expect training time to grow noticeably — SVM training with `probability=True` scales worse than linearly with sample count, so a full-scale run could take several minutes to tens of minutes on a Colab CPU runtime rather than seconds.